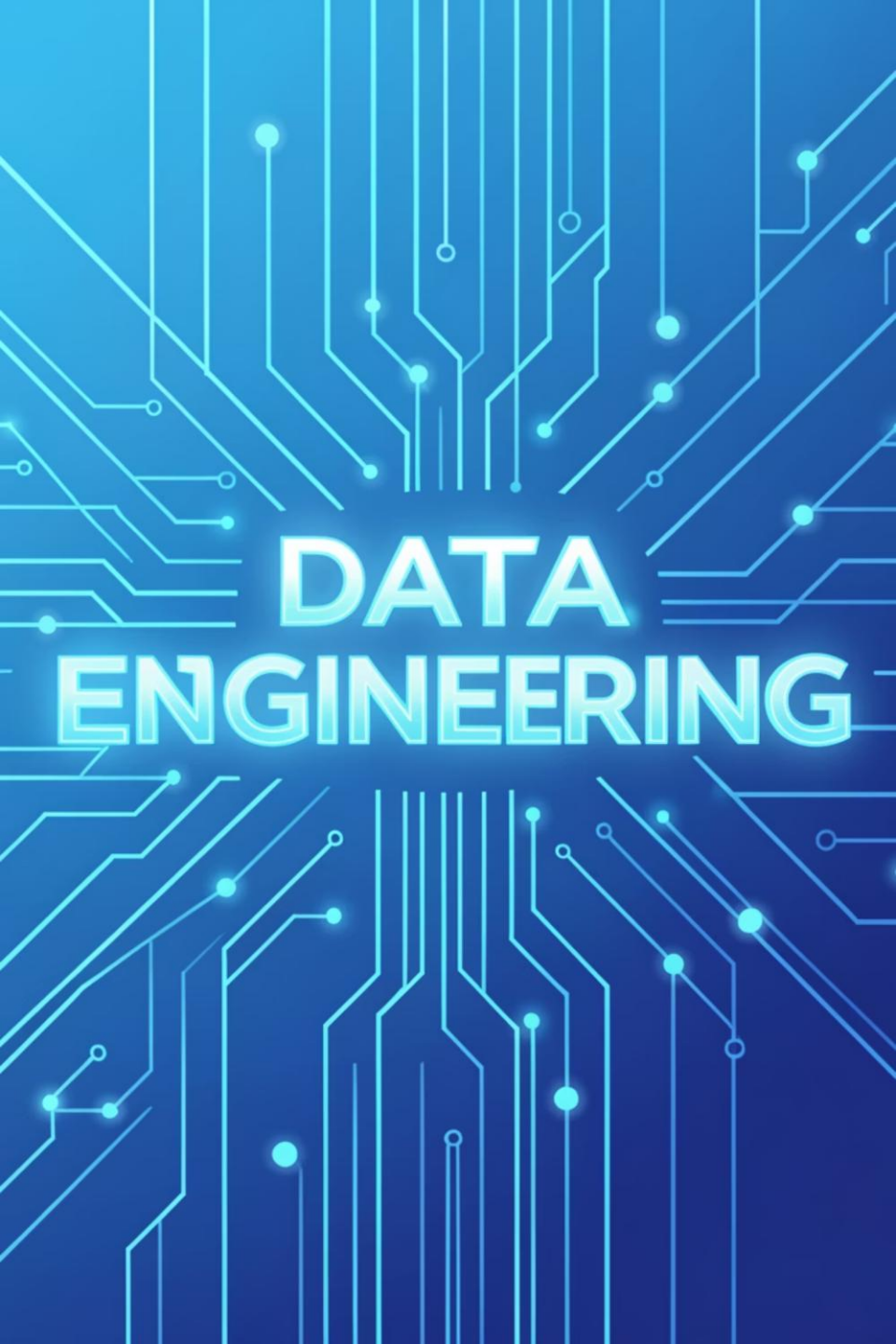




DATA ENGINEERING

Data Engineering Spring '26 – DS471

January 29, 2026

Lecture 5

Logistics: Your Course Essentials

Class Timings

Monday (LH6), Tuesday (LH8): 11:30 AM – 12:20 PM

Friday (LH8): 11:00 AM – 11:50 AM

Office Hours

Monday, Tuesday: 10:00 AM – 11:00 AM, S18 2nd Floor NAB or by appointment.

Credit Hours

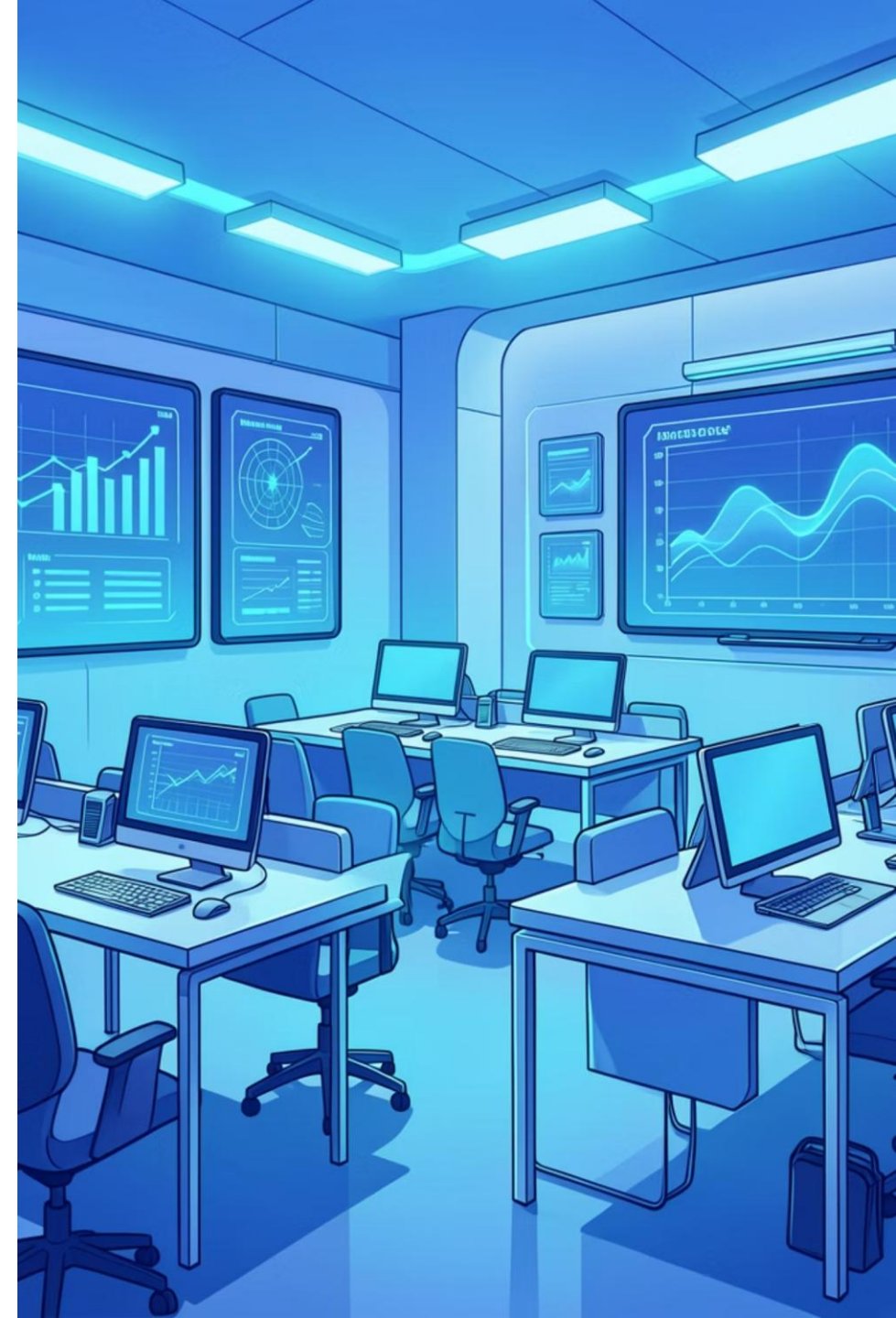
2 hours (Lectures) + 1 hour (Practical Session)

Contact & Support

MS Teams: w5t93yk

Email: farah.saeed@giki.edu.pk

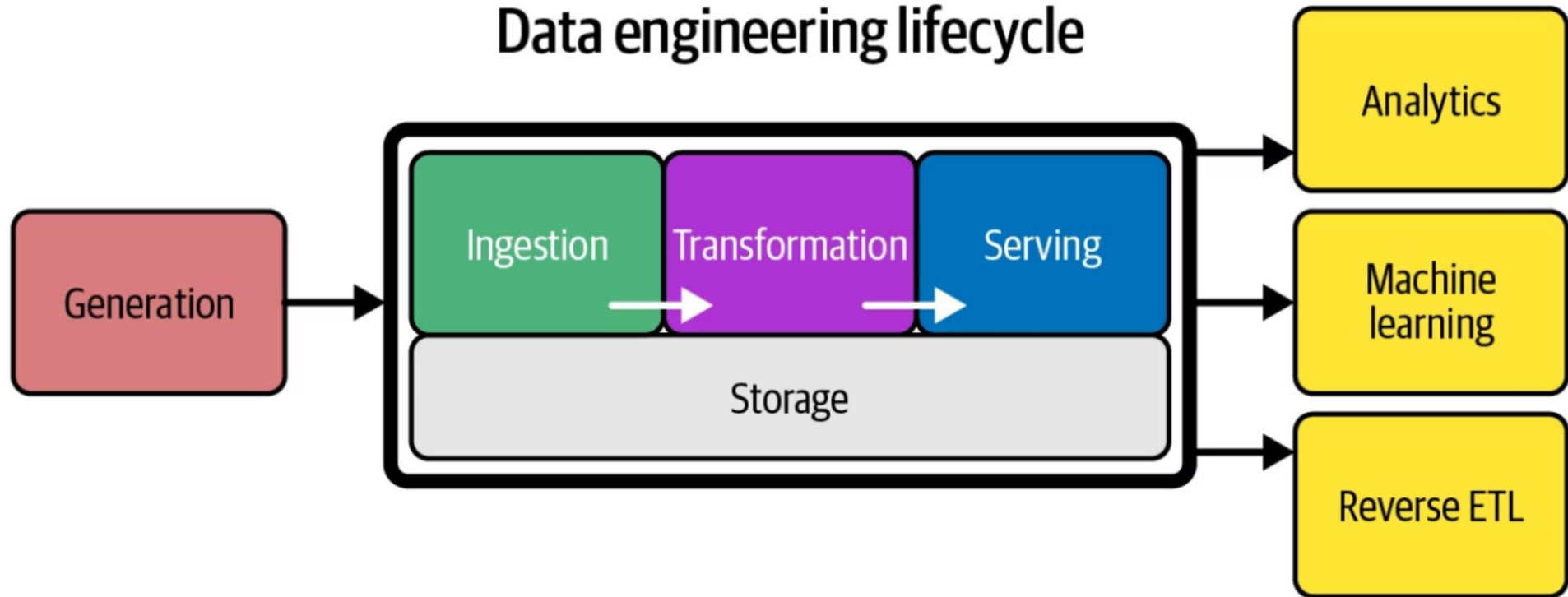
WhatsApp: Reach out for quick queries



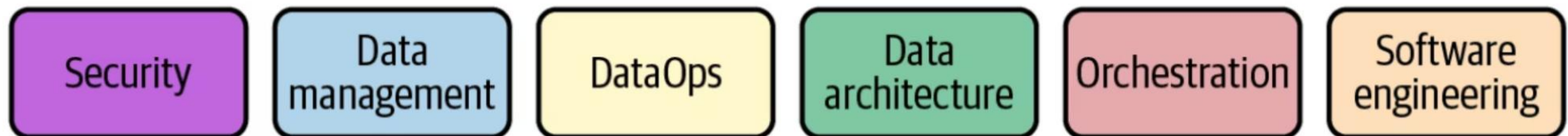
Data Engineering Lifecycle

The data engineering lifecycle comprises stages that turn raw data ingredients into a useful end product, ready for consumption by analysts, data scientists, ML engineers, and others.

Data engineering lifecycle



Undercurrents:



Various stages of the lifecycle may repeat themselves, occur out of order, overlap, or weave together in interesting and unexpected ways.

Generation: Source Systems

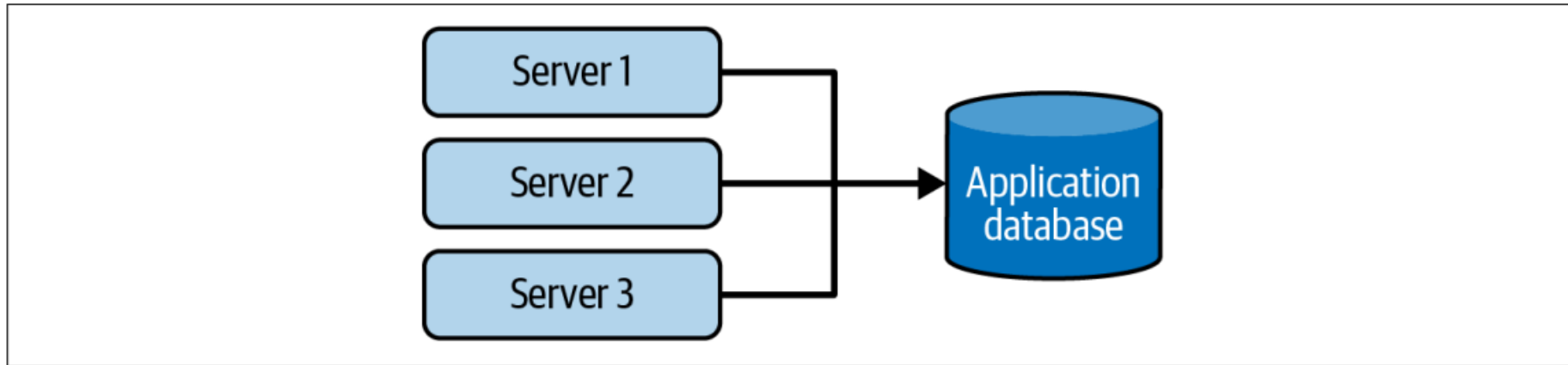


Figure 2-3. Source system example: an application database

Generation: Source Systems

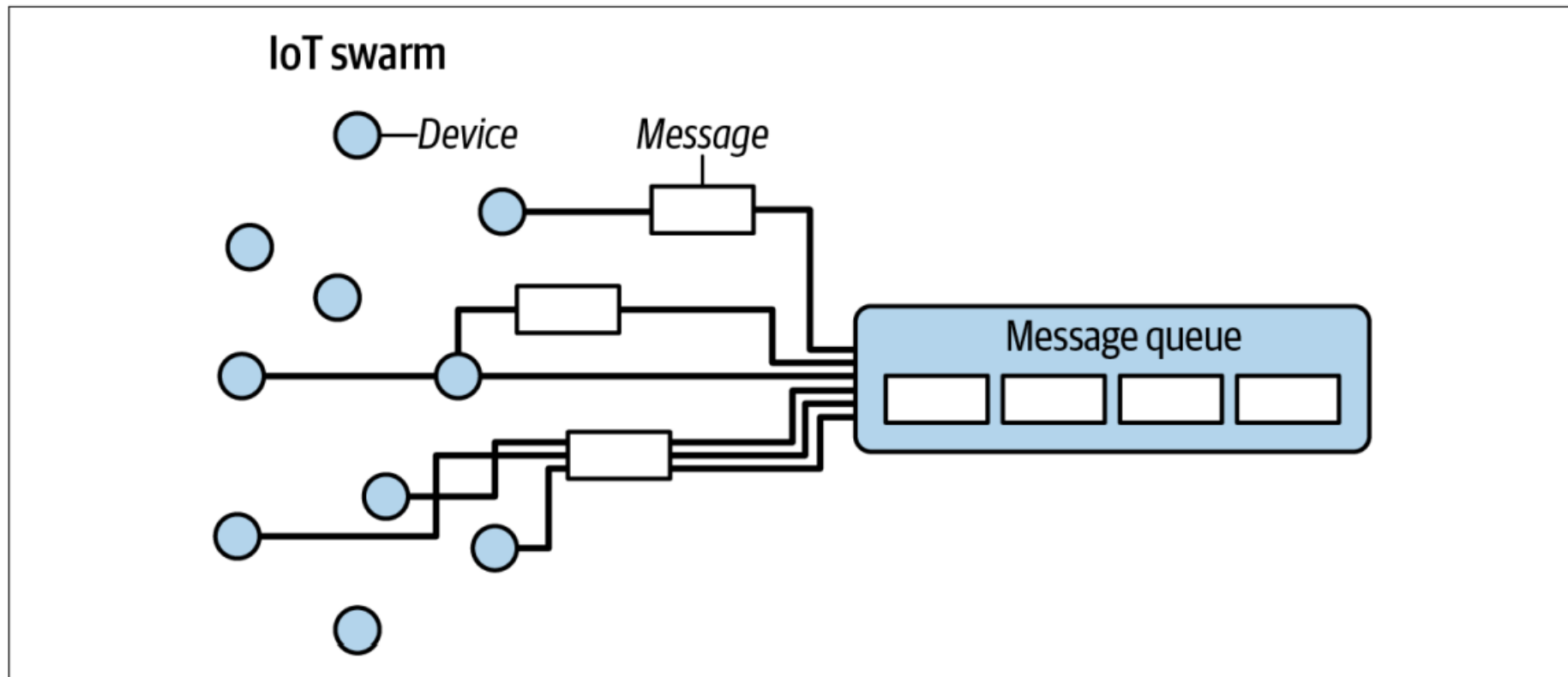


Figure 2-4. Source system example: an IoT swarm and message queue

Storage

Storage runs across the entire data engineering lifecycle, often occurring in multiple places in a data pipeline, with storage systems crossing over with source systems, ingestion, transformation, and serving.

Storage

Data Access Frequency

- **Hot data** is commonly retrieved many times per day, perhaps even several times per second.
- **Lukewarm data** might be accessed every so often—say, every week or month.
- **Cold data** is seldom queried and is appropriate for storing in an archival system.

Ingestion

Unreliable source and ingestion systems have a ripple effect across the data engineering lifecycle.

Ingestion

Key Engineering Considerations

- What are the use cases for the data I'm ingesting? Can I reuse this data rather than create multiple versions of the same dataset?
- Are the systems generating and ingesting this data reliably, and is the data available when I need it?
- What is the data destination after ingestion?
- How frequently will I need to access the data?
- In what volume will the data typically arrive?
- What format is the data in? Can my downstream storage and transformation systems handle this format?
- Is the source data in good shape for immediate downstream use? If so, for how long, and what may cause it to be unusable?
- If the data is from a streaming source, does it need to be transformed before reaching its destination? Would an in-flight transformation be appropriate, where the data is transformed within the stream itself?

Ingestion

Batch vs Streaming

Batch : Batch ingestion is a specialized and convenient way of processing this stream in large chunks—for example, handling a full day's worth of data in a single batch.

Streaming: Streaming ingestion allows us to provide data to downstream systems—whether other applications, databases, or analytics systems—in a continuous, real-time manner. Here, real-time (or near real-time) means that the data is available to a downstream system a short time after it is produced (e.g., less than one second later). The latency required to qualify as real-time varies by domain and requirements.

Ingestion

Batch vs Streaming

Key Engineering Considerations

- If I ingest the data in real time, can downstream storage systems handle the rate of data flow?
- Do I need millisecond real-time data ingestion? Or would a micro-batch approach work, accumulating and ingesting data, say, every minute?
- What are my use cases for streaming ingestion? What specific benefits do I realize by implementing streaming? If I get data in real time, what actions can I take on that data that would be an improvement upon batch?
- Will my streaming-first approach cost more in terms of time, money, maintenance, downtime, and opportunity cost than simply doing batch?

Ingestion

Batch vs Streaming

Key Engineering Considerations

- Are my streaming pipeline and system reliable and redundant if infrastructure fails?
- What tools are most appropriate for the use case? Should I use a managed service (Amazon Kinesis, Google Cloud Pub/Sub, Google Cloud Dataflow) or stand up my own instances of Kafka, Flink, Spark, Pulsar, etc.? If I do the latter, who will manage it? What are the costs and trade-offs?
- If I'm deploying an ML model, what benefits do I have with online predictions and possibly continuous training?
- Am I getting data from a live production instance? If so, what's the impact of my ingestion process on this source system?

Ingestion

Push vs Pull Model

In the **push model** of data ingestion, a source system writes data out to a target, whether a database, object store, or filesystem.

In the **pull model**, data is retrieved from the source system.

Transformation

“Data needs to be changed from its original form into something useful for downstream use cases.”

Key considerations for transformation:

- What's the cost and return on investment (ROI) of the transformation? What is the associated business value?
- Is the transformation as simple and self-isolated as possible?
- What business rules do the transformations support?

Transformation

Drivers of transformation:

- Business logic
- Data featurization for ML

Serving

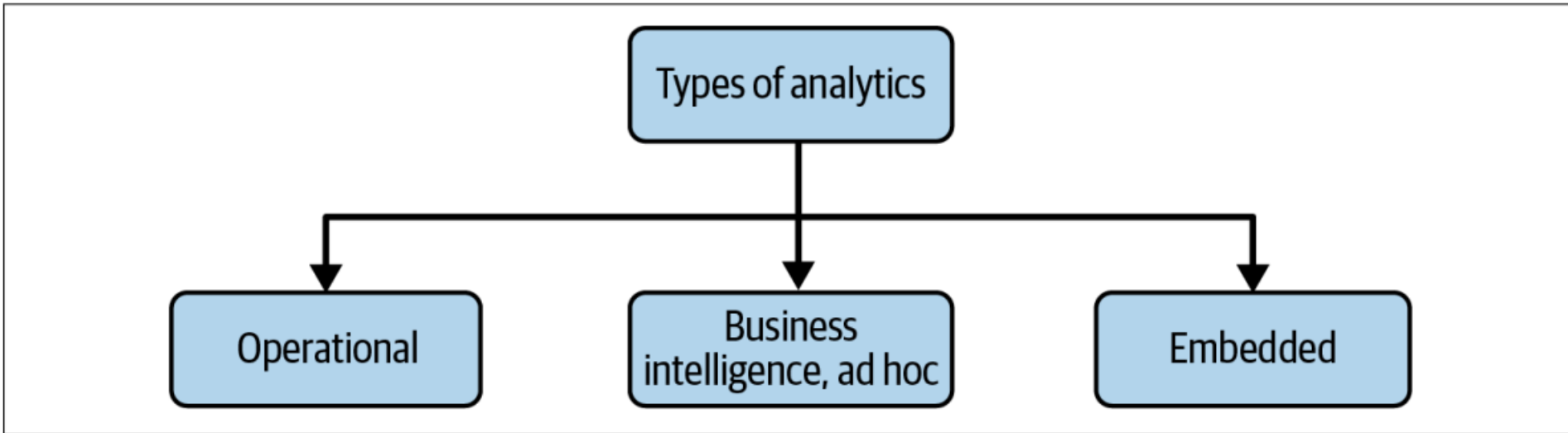


Figure 2-5. Types of analytics

Machine Learning

- Is the data of sufficient quality to perform reliable feature engineering? Quality requirements and assessments are developed in close collaboration with teams consuming the data.
- Is the data discoverable? Can data scientists and ML engineers easily find valuable data?
- Where are the technical and organizational boundaries between data engineering and ML engineering? This organizational question has significant architectural implications.
- Does the dataset properly represent ground truth? Is it unfairly biased?

Reverse ETL

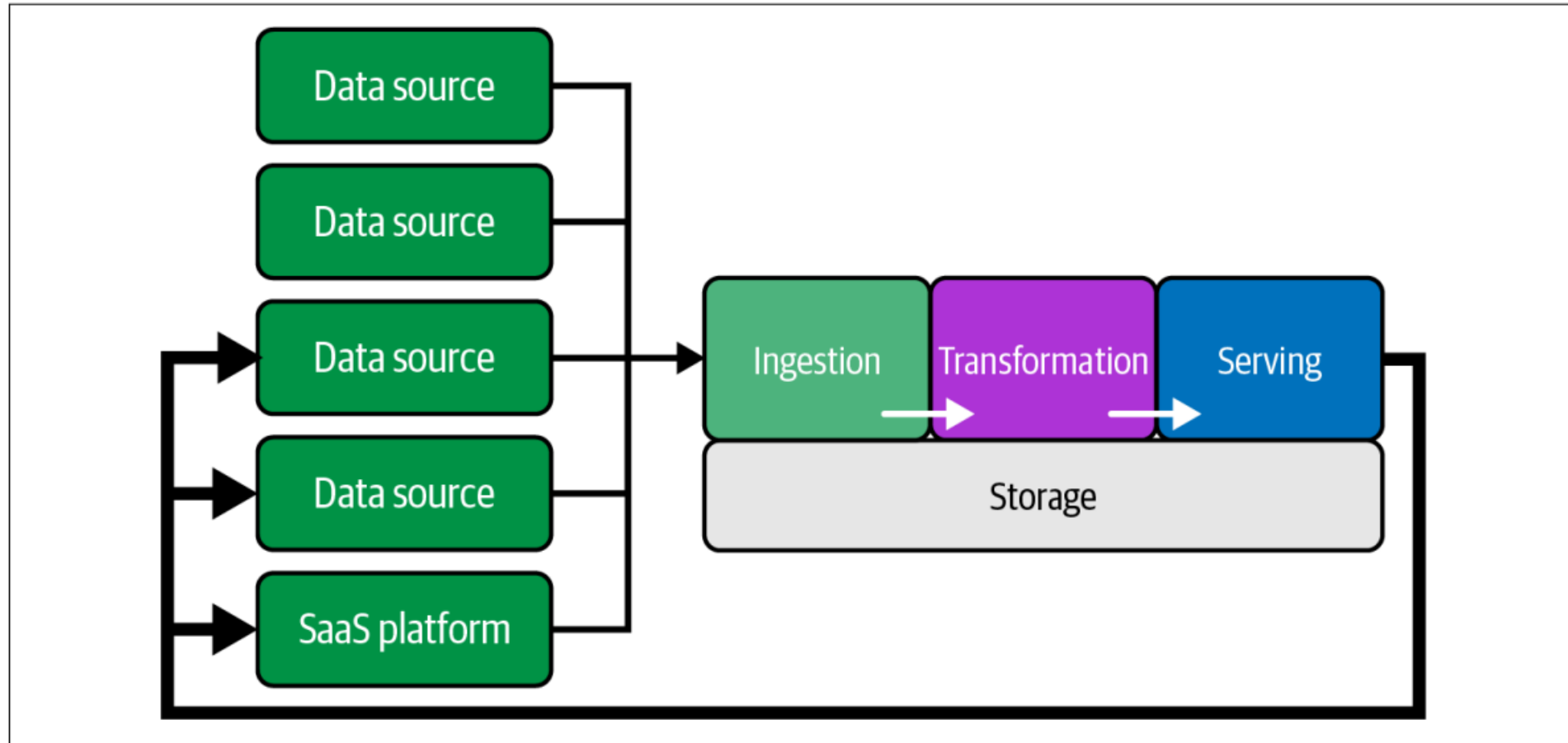


Figure 2-6. Reverse ETL

Undercurrents of Data Engineering

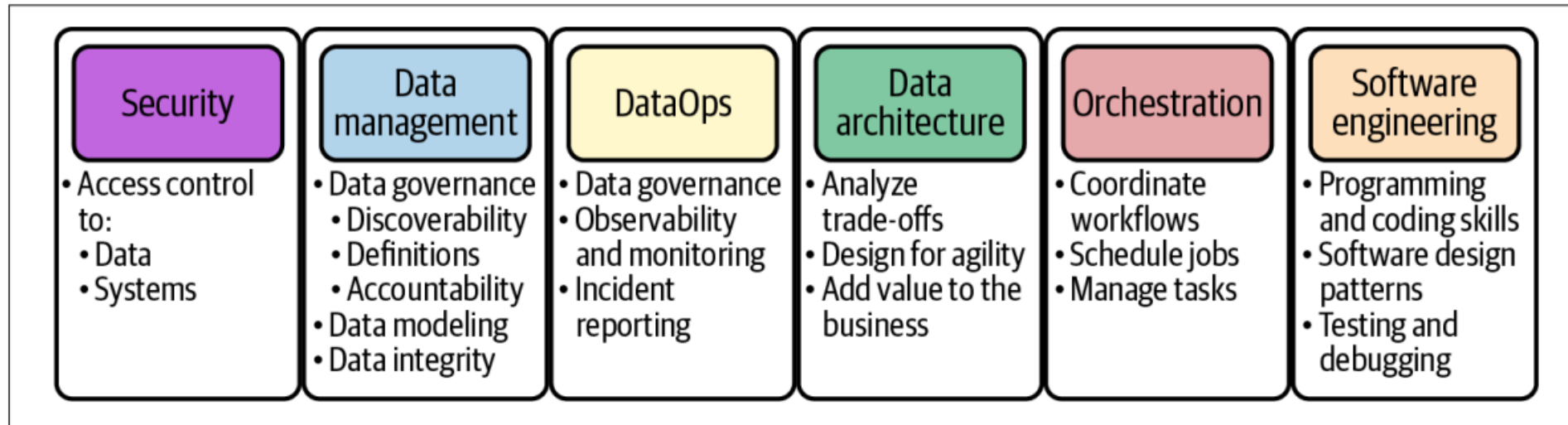


Figure 2-7. The major undercurrents of data engineering

Undercurrents of Data Engineering: Security

- Principle of least privilege

Give users only the access they need to do their jobs today, nothing more.